RustyClean: a Rust pipeline for adaptive minimizer depletion and conditional alignment verification in short-read metagenomics

APPLICATION NOTE

Yufeng Zhang[1], Jing-Yi Zhang[1] and Shi Huang[1,*]

[1]Faculty of Dentistry, The University of Hong Kong, Hong Kong SAR, China

[*]Correspondence: qdu_zyf@163.com

Abstract

Summary: Host-read depletion must retain microbial reads while removing abundant host DNA. RustyClean combines fastp, Deacon minimizer depletion and conditional Bowtie2 verification triggered when ≥30% of reads are removed. On six simulated metagenomes spanning 1–90% host, I/O-optimized RustyClean was 1.99–7.28-fold faster than fastp + Hostile. Paired 100M/90% validation showed 2.12–3.01-fold speedup in all replicates. Batch scaling reached 7.30-fold versus 3.77-fold for Hostile, making RustyClean 1.89-fold faster at eight workers. Verification gave F1 ≥0.998, zero measured host carry-over and ≤0.31% microbial loss on high-host libraries.

Availability and implementation: RustyClean is implemented in Rust, uses fastp, Deacon, Bowtie2 and samtools, and is available under the MIT licence at https://github.com/HuangShiLab/rustyclean. Benchmark source data are available at https://github.com/HuangShiLab/rustyclean-paper.

Supplementary information: Full benchmark tables and figure-generation code are available in the project repository. A detailed benchmark manuscript is available as a separate bioRxiv-ready submission.

Contact: qdu_zyf@163.com

1 Introduction

Short-read metagenomes from host-associated sites often contain abundant host DNA. Alignment-based depletion is sensitive but can remove genuine microbial reads, whereas rapid classifier- or minimizer-based filters are faster but may retain host reads (Gao et al., 2025). Deacon is a minimizer-based sequence filter with near-alignment accuracy and host-fraction-independent speed (Constantinides et al., 2025). However, it is a depletion primitive rather than a complete cohort workflow. RustyClean addresses this gap by adding quality control, adaptive verification, checkpointing, memory-aware concurrency and output validation.

2 Features and implementation

RustyClean AUTO processes each sample in four ordered stages. fastp first performs paired-end adapter detection and quality trimming (Chen et al., 2018). Deacon v0.17.0 then depletes host reads against a compact pangenome index. RustyClean parses the Deacon summary and passes the retained reads to Bowtie2 only when the reported removed fraction is at least 0.30; below this threshold, the Deacon output is accepted without a second alignment pass (Langmead and Salzberg, 2012). Finally, automated checks reject truncated outputs or samples exceeding a configurable residual-host limit.

The workflow is implemented as a single Rust binary and provides sample manifests, atomic checkpoints, content-invalidating input fingerprints, bounded sample-level concurrency and memory-aware worker capping. Workers reuse the same on-disk Deacon index; in standalone batch mode each Deacon process loads this index into its own address space, whereas a resident Deacon server can keep one loaded index available for reuse. In a 16-sample cohort, increasing concurrency from one to eight workers reduced wall time from 2364 s to 302 s, a 7.84× speedup at 98% parallel efficiency, while the median resident set size per Deacon worker remained approximately 4.52 GiB. The streamlined interface intentionally exposes the production AUTO pipeline and a Deacon-only benchmarking mode rather than multiple deprecated depletion backends.

3 Results

On six simulated single-end metagenomes generated with InSilicoSeq and scored with per-read ground truth, RustyClean AUTO required 90.4–2396.7 s across 5–100 million reads (Gourlé et al., 2019). This was 3.45–10.19-fold faster than KneadData v0.12.3 (Constantinides et al., 2023; McIver et al., 2018). The complete fastp+Deacon workflow with verification disabled averaged 141.9–1483.7 s and was 1.05–2.59-fold faster than fastp + Hostile across all six datasets. AUTO with conditional verification was 1.35–2.30-fold faster than fastp + Hostile on the 5–60M datasets, but the added Bowtie2 pass made it 1.29–1.31-fold slower on the two 100M datasets. Thus the large Deacon-versus-Hostile speed advantage is diluted once the shared fastp stage—and, for AUTO, verification—is included. Deacon depletion alone took 9.6–202.4 s and was 40.9–130.2-fold faster than the complete KneadData pipeline. The Hostile comparator is the sequential fastp + Hostile workflow: the plotted times are the sums of independently timed fastp and Hostile stages, not Hostile alone. Across the panel, the Hostile stage alone required 6.17–20.30-fold longer than Deacon depletion only. Peak memory was 4.54 GiB and was bounded by the pangenome index rather than sample size. Deacon backend concurrency scaled near-linearly (Fig. 2a,b).

On four high-host datasets (50–90%), Bowtie2 verification reduced host carry-over to 0.0000% of retained output, whereas fastp + Hostile retained 0.17–0.68% and KneadData retained 0.25% (Table 1). RustyClean AUTO maintained F1 ≥0.998 and lost at most 0.31% of microbial reads. Verification therefore provides a practical route to zero measured host carry-over without applying an alignment pass to every low-host sample. If users prefer the faster fastp+Deacon configuration, disabling recheck preserved F1

≥0.99881 and microbial loss ≤0.238% across the panel, but retained the Deacon host residue (0.0035–0.0037% on high-host libraries).

Reducing intermediate-file compression and using node-local scratch plus a warm Deacon server reversed the 100M runtime penalty. Across the six-dataset panel, the I/O-optimized AUTO pipeline was 1.99–7.28-fold faster than fastp + Hostile. In a same-node, cache-warmed paired validation on 100M/90% data, RustyClean required 428 ± 76 s versus 1032 ± 37 s for fastp + Hostile (n = 3; paired t-test p = 0.006); all three RustyClean replicates were faster, with speedups of 2.12×, 2.26× and 3.01×. RustyClean retained F1 = 0.998408 versus 0.990647 for fastp + Hostile.

Batch concurrency also transferred to the full AUTO pipeline. On 24 balanced 1M-read libraries and a fixed 16-core budget, RustyClean's worker pool reduced wall time from 1027 s at one worker to 141 s at eight workers (7.30×). Under the same worker–thread budget, fastp + Hostile scaled only 3.77× (1002 s to 266 s), so eight-worker RustyClean was 1.89-fold faster than eight-worker fastp + Hostile (Fig. 2c,d). At one worker, RustyClean was within 2.5% of fastp + Hostile (1027 s versus 1002 s); fixed per-sample process start-up, Deacon index loading and orchestration dominate in this sequential setting. Once eight samples were processed concurrently, scheduler overlap and balanced worker/thread allocation gave RustyClean a large throughput advantage.

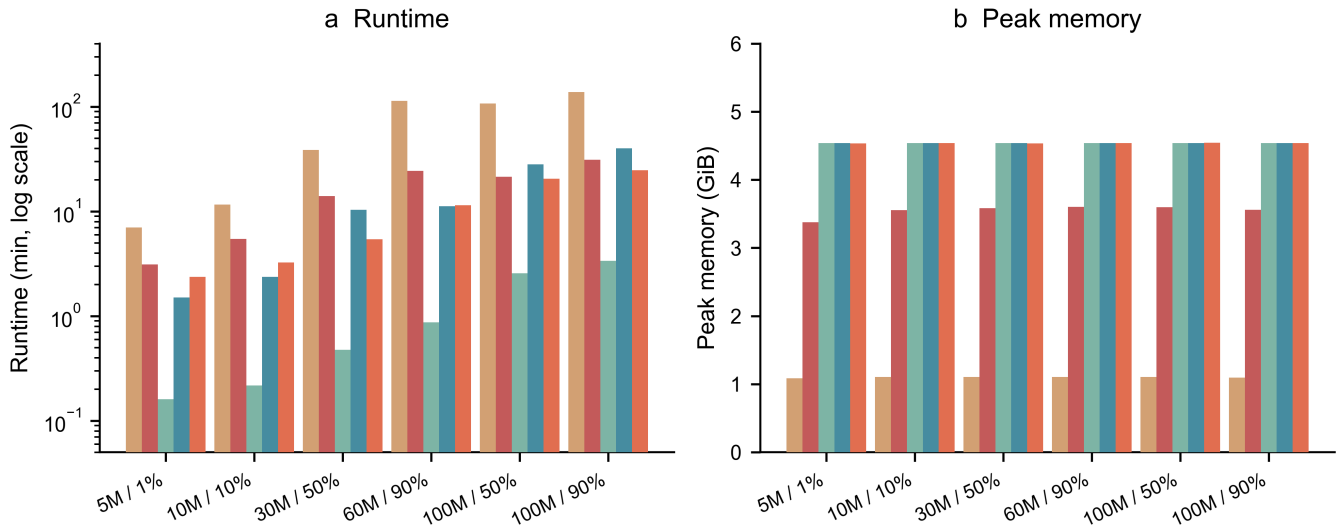


Fig. 1. Performance across simulated metagenomes. (a) Runtime for five depletion workflows or arms; runtime uses a logarithmic scale. KneadData, fastp + Hostile and both RustyClean AUTO arms are full pipelines, whereas Deacon is depletion only. The Hostile + fastp bar is the sum of separately timed fastp and Hostile stages. (b) Peak memory.

Peak per-process memory remained nearly flat across worker counts: 4.54 GiB for RustyClean at both one and eight workers, versus 3.58 and 3.49 GiB for the largest Hostile process. These values are the largest process in each pipeline rather than the sum over concurrent workers. In standalone RustyClean mode, each Deacon worker loads the same index path into its own process; the operating system can cache index bytes on disk, but the deserialized in-memory index is not shared. RustyClean also supports a Deacon server mode (`--use-deacon-server --deacon-server-dir`) in which one resident server loads the index once and client runs reuse it, reducing duplicate index memory and index-load time in cohort mode.

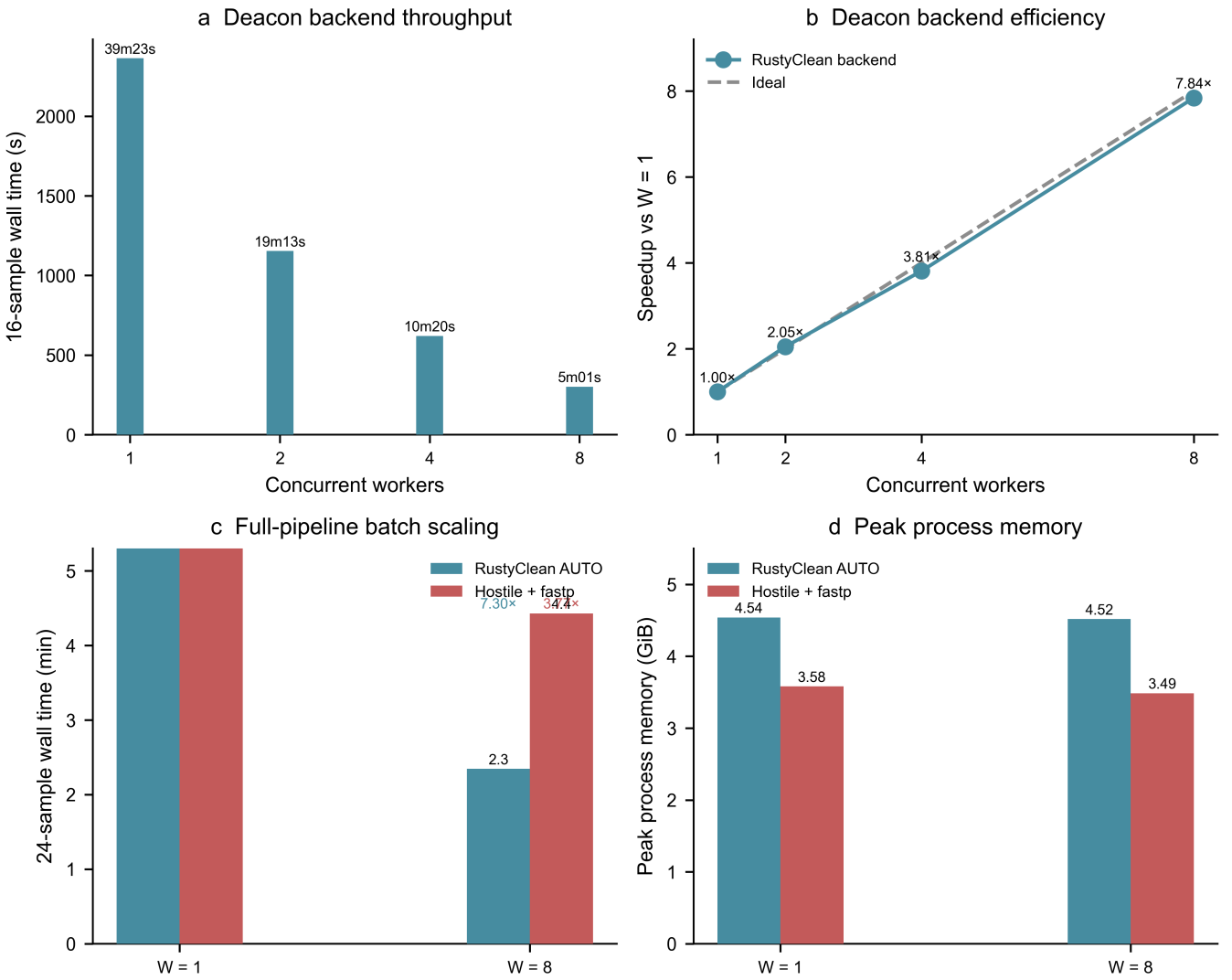


Fig. 2. Parallel scaling and resource use. (a) Cohort wall time for sixteen 10M-read samples processed by the Deacon backend with one to eight workers. (b) Deacon backend speedup and parallel efficiency. (c) Full-pipeline wall time for 24 balanced 1M-read libraries with RustyClean AUTO versus fastp + Hostile at one and eight workers; annotations show W = 8 scaling. (d) Peak resident set size of the largest pipeline process; this is not the summed memory of all concurrent workers.

Table 1. Accuracy on four high-host simulated datasets.

Workflow	F1 range	Microbial loss (%)	Host carry-over (%)
KneadData	0.97603–0.99098	1.425–2.154	0.2493–0.2500
fastp + Hostile	0.96522–0.99878	0.000–0.039	0.1734–0.6758
Deacon only	0.99920–0.99997	0.000–0.121	0.0035–0.0037
RustyClean AUTO	0.99845–1.00000	0.000–0.310	0.0000
RustyClean AUTO without recheck	0.99920–0.99997	0.0055–0.1211	0.0035–0.0037

Table 2. Same-node paired validation of an individual 100M / 90% sample (n = 3 runs).

Replicate	RustyClean AUTO (s)	fastp + Hostile (s)	Speedup	RustyClean F1	fastp + Hostile F1
1	499.02	1060.01	2.12×	0.998408	0.990647
2	437.35	990.14	2.26×	0.998408	0.990647
3	347.56	1046.96	3.01×	0.998408	0.990647
Mean ± SD	427.98 ± 76.16	1032.37 ± 37.15	2.47×	0.998408	0.990647

Paired t-test on runtime: p = 0.0061. Wilcoxon signed-rank test is bounded at p = 0.25 for n = 3.

Table 3. Full-pipeline batch parallelization on 24 balanced 1M-read libraries (16 cores).

Workflow	W = 1 (s)	W = 8 (s)	Scaling	Peak process RSS at W = 8 (GiB)
RustyClean AUTO	1027.14	140.67	7.30×	4.52
fastp + Hostile	1002.15	265.62	3.77×	3.49

At eight workers, RustyClean AUTO was 1.89× faster than fastp + Hostile. Peak process RSS is the largest single process, not the summed memory of all concurrent workers.

4 Conclusion

RustyClean turns rapid minimizer depletion into a reproducible host-depletion workflow. In same-node paired single-sample validation, optimized RustyClean was 2.12–3.01-fold faster than fastp + Hostile and retained higher F1. In multi-sample batch mode, W=8 RustyClean was 1.89-fold faster than W=8 fastp + Hostile. Conditional Bowtie2 verification preserves the speed advantage of Deacon while removing residual host signal precisely in the high-host samples where it matters most.

Availability and implementation

RustyClean is available under the MIT licence at https://github.com/HuangShiLab/rustyclean. The benchmarked research source is archived at commit `7ab1a4b`; the streamlined AUTO interface is at commit `55f93af`. Figure source data and analysis code are available at https://github.com/HuangShiLab/rustyclean-paper.

Acknowledgements

The authors thank colleagues at The University of Hong Kong for discussions and computational support.

References

Chen S. *et al.* (2018) fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics*, 34, i884–i890.

Constantinides B. *et al.* (2023) Hostile: accurate decontamination of microbial host sequences. *Bioinformatics*, 39, btad728.

Constantinides B. *et al.* (2025) Deacon: fast sequence filtering and contaminant depletion. *bioRxiv*, 2025.06.09.658732.

Gao Y. *et al.* (2025) Benchmarking short-read metagenomics tools for removing host contamination. *GigaScience*, 14, giaf004.

Gourlé H. *et al.* (2019) Simulating Illumina metagenomic data with InSilicoSeq. *Bioinformatics*, 35, 521–522.

Langmead B. and Salzberg S.L. (2012) Fast gapped-read alignment with Bowtie 2. *Nature Methods*, 9, 357–359.

McIver L.J. *et al.* (2018) bioBakery: a meta'omic analysis environment. *Bioinformatics*, 34, 1235–1237.